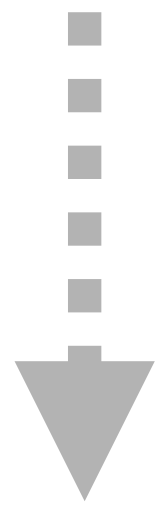
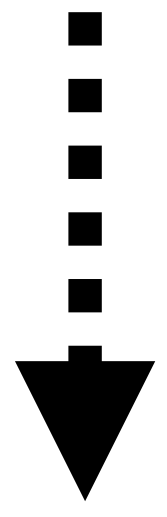
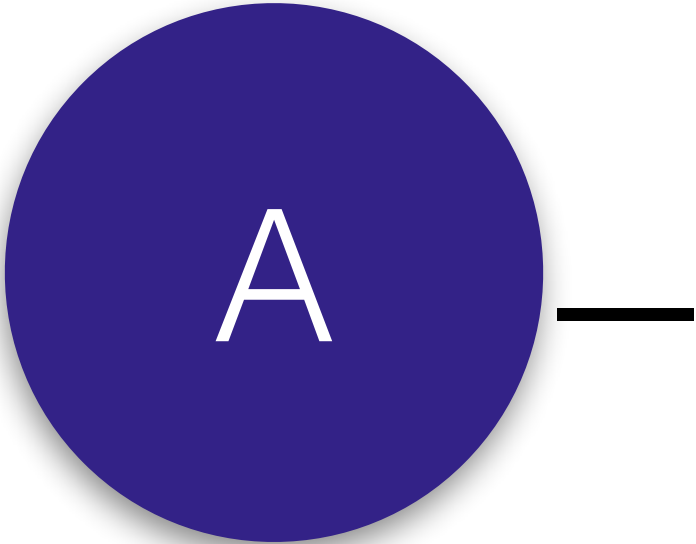



A




A

When executed concurrently, the update behaviours and logging behaviours should agree on the order of updates.

# There are restrictions we want to keep

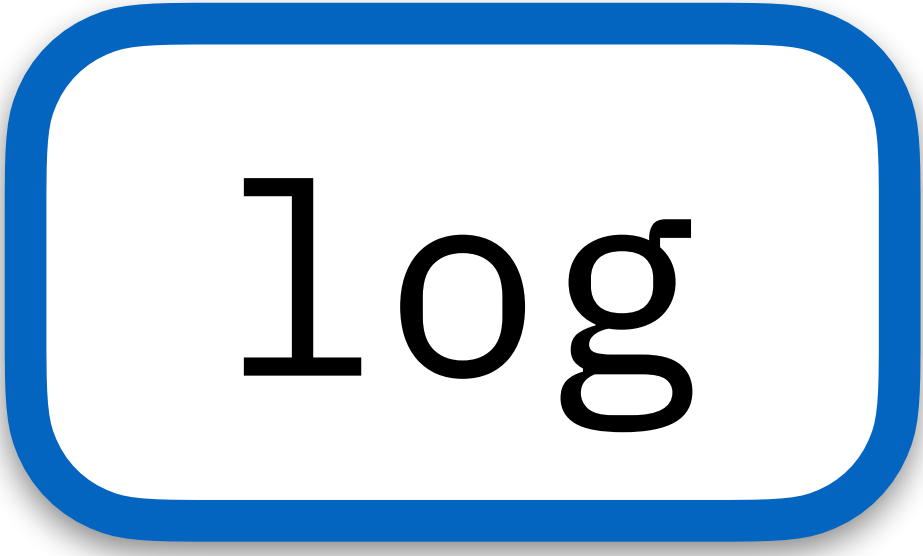
38

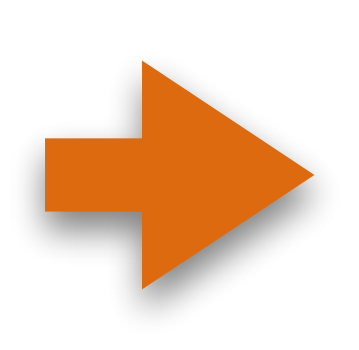
```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

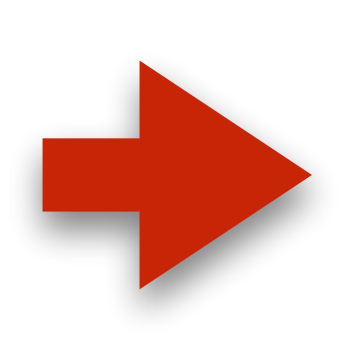
```
update(src, +100)
```

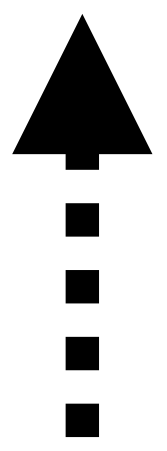
```
update(src, -100)
```

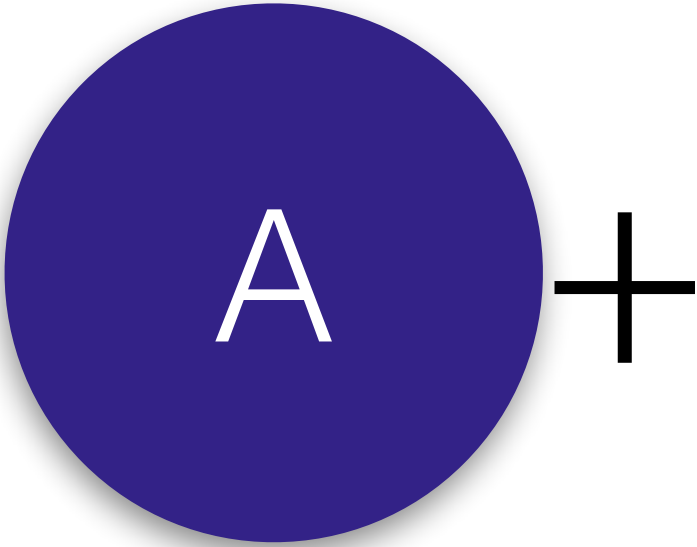
account

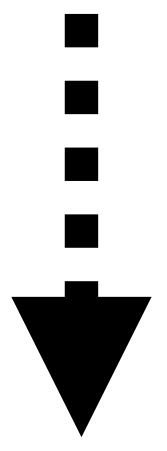

log

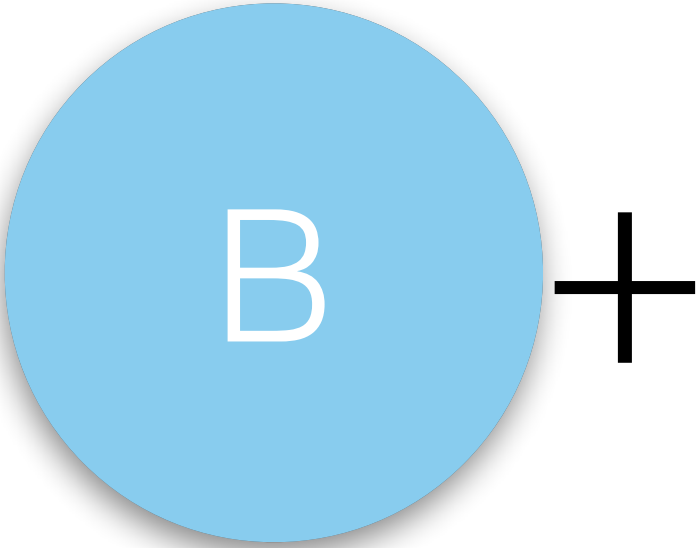


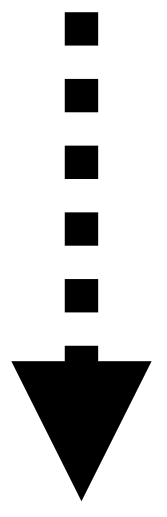


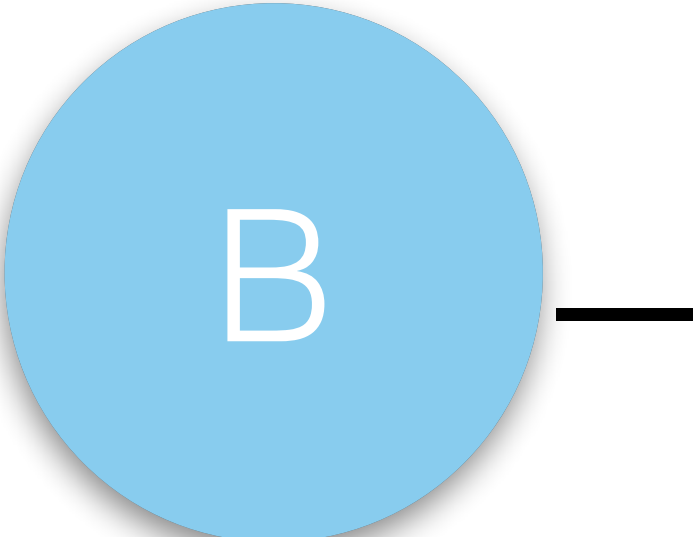


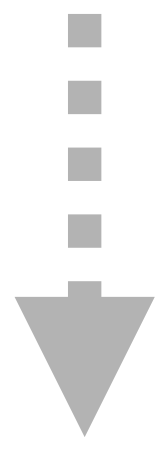

A
+

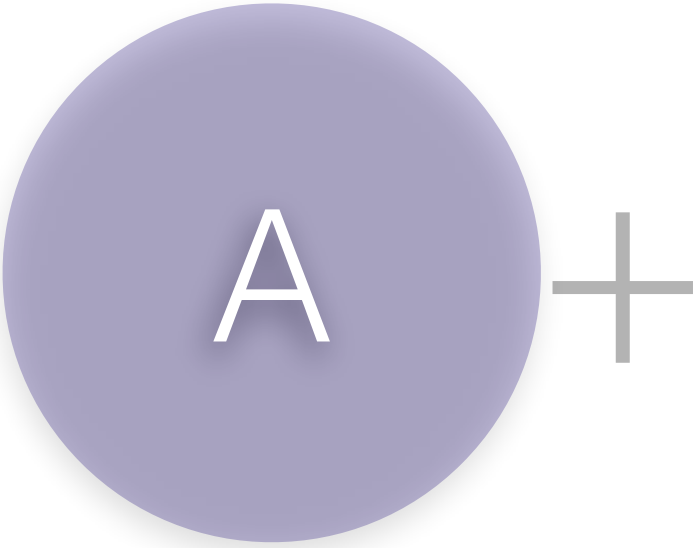



B
+




B




A

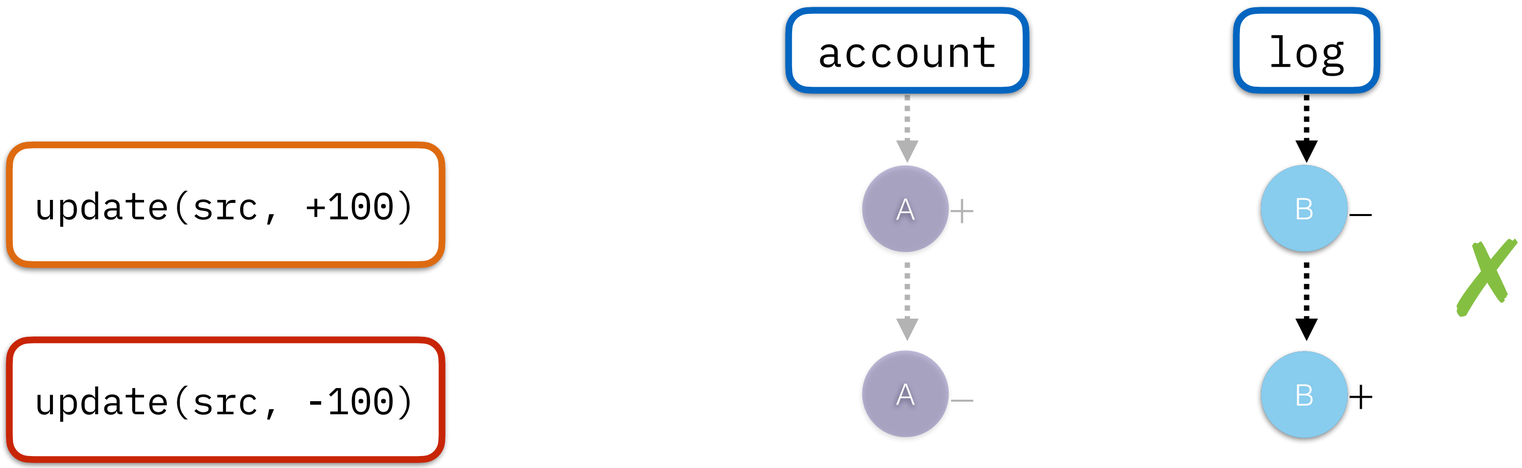
In this case, the semantics provide the appropriate restrictions.

# There are restrictions we want to keep

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

When executed concurrently, the update behaviours and logging behaviours should agree on the order of updates.

In this case, the semantics provide the appropriate restrictions.



# There are restrictions we want to keep

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

In this case, the semantics provide the appropriate restrictions.

| Order |
| --- |
| Permitted |